RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models

YunDa Tsai*
yundat@nvidia.com
NVIDIA

Mingjie Liu*
mingjiel@nvidia.com
NVIDIA

Haoxing Ren
haoxingr@nvidia.com
NVIDIA

ABSTRACT

This paper presents **RTLFixer**, a novel framework enabling automatic syntax errors fixing for Verilog code with Large Language Models (LLMs). Despite LLM's promising capabilities, our analysis indicates that approximately 55% of errors in LLM-generated Verilog are syntax-related, leading to compilation failures. To tackle this issue, we introduce a novel debugging framework that employs Retrieval-Augmented Generation (RAG) and ReAct prompting, enabling LLMs to act as autonomous agents in interactively debugging the code with feedback. This framework demonstrates exceptional proficiency in resolving syntax errors, successfully correcting about 98.5% of compilation errors in our debugging dataset, comprising 212 erroneous implementations derived from the VerilogEval benchmark. Our method leads to 32.3% and 10.1% increase in pass@1 success rates in the VerilogEval-Machine and VerilogEval-Human benchmarks, respectively. The source code and benchmark are available at https://github.com/NVlabs/RTLFixer.

ACM Reference Format:
YunDa Tsai, Mingjie Liu, and Haoxing Ren. 2024. **RTLFixer**: Automatically Fixing RTL Syntax Errors with Large Language Models. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3657353

1 INTRODUCTION

Large language models (LLMs) present great promise in automating hardware design, especially in their capacity to understand design intentions and produce Verilog code from natural language [8]. Recent efforts, such as VeriGen [17] and VerilogEval [9], have primarily focused on zero-shot code generation. However, akin to numerous complex programming tasks, generating flawless code in a single attempt poses a significant challenge, with a high likelihood of errors. Consequently, there exists a clear need for robust debugging and refinement capabilities in Verilog code generated by Large Language Models. This need arises from that like human programmers, achieving precision often requires multiple iterations.

Most importantly, it is evident that Large Language Models encounter challenges in generating fully syntactically correct Verilog code. Surprisingly, our analysis reveals that a substantial 55% of the errors generated by LLMs for Verilog code are comprised of syntax errors, surpassing the occurrence of logic errors detected through simulation. Rectifying syntax errors not only enhances the overall accuracy of LLM-generated code but also holds the potential to alleviate manual efforts for human engineers engaged in Verilog coding. The recognition and mitigation of syntax errors stand as imperative steps, not only for refining LLM capabilities but also for streamlining the coding process for human practitioners in the domain of Verilog development.

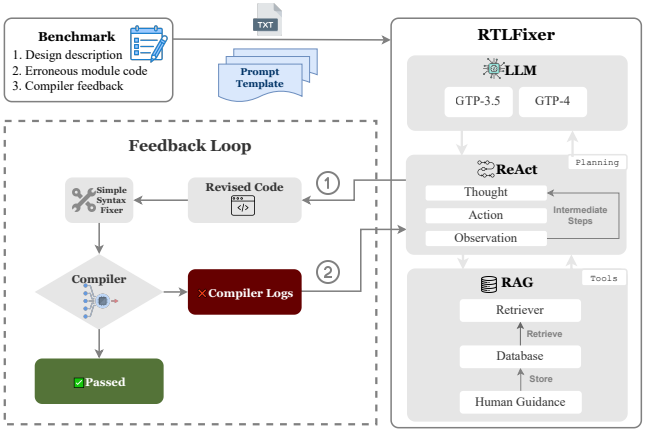


Figure 1: Overview of RTLFixer. The Autonomous Language Agent fixes the syntax error via a feedback loop. ReAct handles the iterative code refinement with intermediate reasoning and action steps. Human expert guidance is incorporated through RAG.

Despite such challenges, LLMs have showcased remarkable capabilities in reasoning and enhancing action plans to address exceptions. The ReAct framework [20] integrates reasoning and action synthesis in language models, demonstrating LLM's capability to engage in reasoning processes and refine decision-making through interactive feedback. Similarly, SelfDebug [3] and SelfEvolve [6] illustrate the model's ability to self-identify mistakes by scrutinizing execution results and articulating generated code in natural language. It is essential to highlight that prior works do not explicitly address the correction of syntax errors and primarily center on improving the accuracy of generated code, particularly in Python, where language models excel syntactically.

On the other hand, Large Language Models are acknowledged for their inclination to produce factual errors, a phenomenon termed hallucination [5]. To mitigate this challenge, the Retrieval-Augmented Generation (RAG) paradigm [7] has been introduced, integrating retrieval mechanisms to improve the precision of generated content by incorporating information from external knowledge sources.

*equal contribution

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3657353

In this paper, we introduce **RTLFixer**, a innovative debugging framework that utilizes LLMs as autonomous language agents in conjunction with RAG. We exclusively focus on addressing the challenge of rectifying syntax errors in RTL Verilog code—an essential problem with potential benefits for both LLMs and human engineers. As shown in Figure 1, our framework combines established human expertise stored in a retrieval database for correcting syntax errors while simultaneously harnessing the capabilities of LLMs as autonomous agents for reasoning and action planning (ReAct). Through incorporating human expertise, our approach provides explicit guidance and explanations when LLMs face challenges in error correction. The stored compiler messages and human expert guidance function as a persistent external non-parametric memory database, enhancing results through RAG. By empowering LLMs with ReAct, LLMs serve as autonomous agents adept at strategically planning intermediate steps for iterative debugging. We also create VerilogEval-syntax, a Verilog syntax debugging dataset, derived from VerilogEval [9], containing 174 erroneous implementations.

Our contributions are summarized as follows:

- Our framework demonstrates an impressive 98.5% success rate in resolving syntax errors, resulting in a noteworthy 32.3% and 10.1% improvement in the pass@1 metrics achieved solely by addressing syntax errors in VerilogEval-Machine and VerilogEval-Human benchmarks, respectively.
- Our framework also improves the syntax success rate from 73% to 93% on the RTLLM benchmark [11], demonstrating the generalizability of this approach.
- Compared to One-shot generation, ReAct enhances syntax success rates by 25.7%, 26.4%, and 31.2% with iterative feedback from Simple, iverilog, and Quartus, respectively.
- RAG with human guidance significantly improves syntax success rates, up to 31.2% and 18.6% with feedback from Quartus for One-shot and ReAct prompting, respectively.

The remainder of this paper is structured as follows. In Section 2, we present preliminary works on LLMs for Verilog code generation, ReAct, and RAG. Our debugging framework **RTLFixer** is elucidated in Section 3, where we empower LLMs as autonomous agents with ReAct and innovatively provide human guidance through RAG. Section 4 details our experimental results, showcasing the effectiveness of our method in correcting syntax errors and improving the pass rate. Finally, Section 6 summarizes and concludes the paper.

2 PRELIMINARIES

In this section, we begin by briefly exploring the advancements and applications of LLMs for Verilog code generation in Section 2.1. We then delve into the synthesis of reasoning and action in LLMs, elaborated in Section 2.2. Finally, we discuss Retrieval-Augmented Generation in Section 2.3.

2.1 LLMs for Verilog Code Generation

Large Language Models exhibit the capability to generate code, with Codex [2] standing as an early exemplar. GitHub Copilot [4], building upon such groundwork, played a crucial role in pioneering LLM-based code completion engines, contributing significantly to the domains of auto-completion and conversational code generation. DAVE [14] emerged as an early study of LLM tailored for hardware design. VeriGen [17] further expanded the dataset scope and experimented with open-sourced models. Following suit, ChipChat [1], leveraging GPT-4, demonstrated the extensive potential of LLMs in collaboratively generating processors and other hardware designs. In parallel, benchmarks such as VerilogEval [9] and RTLLM [11] played a pivotal role in advancing the application of LLMs in Verilog code generation.

2.2 Reasoning and Action Synthesis of LLMs

Large Language Models (LLMs) have demonstrated proficiency in both reasoning and planning across various tasks. In terms of reasoning, methods like chain-of-thought [18] empower LLMs to break down intricate problems into logical steps, significantly enhancing their problem-solving abilities. LLMs also excel in interactive decision-making and formulating action plans, effectively leveraging digital tools, as shown in works such as ToolLLM [15].

The ReAct [20] framework represents a notable advancement in LLM capabilities by seamlessly integrating reasoning and action planning. This framework enables LLMs to generate both reasoning traces and specific actions, facilitating dynamic interaction with external information sources. This integration not only enhances LLM's performance in complex tasks but also renders them more reliable and versatile as autonomous agents, capable of delivering more accurate and context-aware responses.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [7] represents a significant advancement in addressing the limitations of Large Language Models when handling knowledge-intensive tasks. LLMs despite containing extensive factual knowledge, often encounter challenges in accessing and effectively manipulating this information. RAG leverages a combination of LLMs, which serve as parametric memory, and an external knowledge base, such as Wikipedia, functioning as non-parametric memory. This unique approach allows RAG to access and retrieve relevant documents or passages from the external knowledge base based on the input query. Consequently, this enriches the context available to the text generator, resulting in outputs that exhibit improved accuracy and factual consistency. In the context of code generation, several works such as ReACC [10] and RepoCoder [21] have successfully harnessed the capabilities of RAG to enhance the code generation proficiency of LLMs, showcasing its transformative potential.

3 RTLFIXER: RESOLVING SYNTAX ERROR WITH LLM AGENTS AND RETRIEVAL

In this section, we explain the details of **RTLFixer**, which utilizes Autonomous Language Agents enhanced with ReAct and Retrieval-Augmented Generation (RAG). The framework's structure is outlined in Figure 1 (Section 3.1), and the application of ReAct is thoroughly discussed in Section 3.2. Furthermore, Section 3.3 explores the integration of human expert guidance using RAG. Finally, we explain the curation process for our VerilogEval-syntax error dataset.

3.1 Overview of RTLFixer

RTLFixer comprises an LLM for code generation, RAG for accessing human expert guidance, and ReAct for improved task decomposition, tool use, and planning. Our approach starts by formulating

an input prompt integrating a benchmark dataset problem into a template, followed by the agent utilizing RAG and ReAct, revising erroneous Verilog code. If syntax errors persist, error logs from the compiler as well as retrieved human guidance from the database are provided as feedback. This interactive debugging loop can be repeated multiple times until all errors are resolved.

3.2 Reasoning and Action Planning through ReAct Iterative Prompting

We enable Large Language Models to function as autonomous agents for reasoning and action planning through the **ReAct** prompting mechanism [20]. In ReAct, LLMs generate both reasoning traces and task-specific actions in an interleaved manner. The input prompt, along with the ReAct instruction prompt, is provided to an LLM. Subsequently, the LLM initiates the generation of ReAct steps, each consisting of Thought, Action, and Observation components. An example of a ReAct instruction prompt is depicted in Figure 2b, while Figure 2c illustrates the self-prompting process, showcasing the intermediate steps within each iteration of ReAct.

During this process, the LLM prompts itself for thoughts on how to address the error and selects the next action. Potential actions include generating an explanation for the error, searching for a solution in the human expert guidance database, revising the code, and submitting the revised code to the compiler, among other possibilities. The output of the chosen action becomes the observation in the prompt. The agent continues prompting until the compilation is successful, selecting the Finish action to output the final response. If unsuccessful, the process iterates up to n times, where n is a user-selected hyperparameter. Our objective is to assess the effectiveness of a fully automated feedback-driven solution.

We employ **One-shot** prompting, illustrated in Figure 2a, as the baseline for comparing with ReAct. One-shot prompting involves only a single-turn of feedback from the compiler message, but it excludes ReAct prompting, which decomposes syntax fixing through reasoning and action planning, and the iterative multi-round interactions with the compiler.

3.3 Retrieval Augmented Generation (RAG)

We leverage Retrieval-Augmented Generation (RAG), a potent technique that notably enhances Large Language Models' capabilities by incorporating human expert guidance through a retriever. A key distinction from traditional RAG lies in our curated database, enriched with human instructions and demonstrations.

The retrieval database curation process involves a meticulous procedure of categorizing syntax errors and developing instructions and demonstrations for syntax error resolution. In the initial step, we categorize various syntax errors into groups using error number tags provided by compilers (such as Quartus) in the compiler logs. During the manual inspection of LLM's struggle cases, it becomes evident that ambiguous error messages present a significant challenge, impeding the model's error resolution capabilities. The inclusion of clear instructions, and demonstrations of possible solutions enables the LLM to adeptly address errors. To facilitate this, human experts offer detailed explanations for compiler logs, serving as human expert guidance. An illustrative example of common errors is showcased in Figure 3. Subsequently, all compiler logs,

System Prompt:
Implement the Verilog module based on the following description. Assume that signals are positive clock/clk edge triggered unless otherwise stated.

Problem Description:
Implement the Verilog module based on the following description. Assume that signals are positive clock/clk edge triggered unless otherwise stated.
Given an 8-bit input vector [7:0], reverse its bit ordering.

```
module top_module (
input [7:0] in,
output [7:0] out
);
```

Erroneous Implementation:

```
assign {out[0],out[1],out[2],out[3],out[4],out[5],out[6],out[8]} = in;
endmodule
```

Feedback:
main.v:5: error: Index out[8] is out of range.
1 error(s) during elaboration.

(a) One-shot Prompt Template

Solve a task with interleaving Thought, Action, Observation steps.
Thought can reason about the current situation, and Action can be the following types:
(1) Compiler[code], which compiles the input code and provide error message if there is syntax error.
(2) Finish[answer], which returns the answer and finished the task.
(3) RAG[logs], input the compiler log and retrieve expert solutions to fix the syntax error.

(b) ReAct Instruction (System Prompt)

Question:
What is the syntax error in the given Verilog module implementation and how to fix it?

Thought 1:
Implement a finite state machine that searches for message boundaries in a byte stream.
Action 1: Compiler

```
always @ (state or in) begin
case (state)
 4'b0001: begin
```

Observation 1:
main.v:15: error: out is not a valid l-value in top_module.
Thought 2:
The code has a syntax error because the "out" signal was declared as a wire instead of a register. I need to correct this and run the compilation.
Action 2: RAG[..not a valid l-value..]
Use assign statements instead of always block if possible.
...
Action n: Finish
The revised code is as follow: ...

(c) ReAct Steps with Thought-Action-Observation.

Figure 2: Prompts used for ReAct. (a) shows the One-shot prompting template with feedback message. (b)-(c) demonstrate the example where LLMs serve as autonomous agents with ReAct to decompose syntax fixing problems with reasoning and planning.

error code segments, and corresponding human guidance undergo systematic storage in the database for future retrieval.

We integrate the human guidance and demonstration database with Large Language Models (LLMs) using Retrieval-Augmented Generation (RAG). RAG enables the retrieval of pertinent documents or data from a source, utilizing this information as context for the original input prompt. This approach allows the language model to access the latest information without the need for retraining, proving especially valuable in enhancing the model's capacity to generate more accurate and reliable outputs. Leveraging RAG

Compiler Logs:
Object 'clk' is not declared. Verify the object name is correct. If the name is correct, declare the object.
Human Expert Guidance:
Check if 'clk' is an input. If not, and if 'clk' is used within the module, make sure the name is correct. If it's meant to trigger an 'always' block, replace 'posedge clk' with '*'.

Compiler Logs:
Index cannot fall outside the declared range for vector
Human Expert Guidance:
Carefully examine the index values to prevent encountering 'index out of bound' errors in your code. When utilizing parameters for indexing, try to use binary strings for performing the indexing operation instead.

Figure 3: Examples of common error categories that LLM constantly could not solve and the corresponding human expert guidance in the retrieval database.

further ensures that the language model has access to the most current and relevant information, including compiler logs and human guidance, facilitating effective error resolution.

Figure 3 illustrates two common error categories along with a demonstration of compiler logs and corresponding human guidance. For this task, common retrievers such as pattern-matching, fuzzy search, or similarity search with a vector database are suitable. In our experiments, we opted for an exact match to error tags for simplicity, given the limited number of error cases. We collected 7 common error categories with 30 entries for iverilog and 11 common error categories with 45 entries for Qaurtus in total.

3.4 Debugging Dataset

We created a novel benchmark dataset, VerilogEval-syntax, based on the VerilogEval benchmark [9]. This dataset comprises flawed code implementations sourced from the VerilogEval problem set. Each entry includes the original problem description and erroneous implementation containing syntax errors.

The dataset curation includes sampling, filtering, and clustering. Code samples were selected from VerilogEval problems using One-shot and ReAct prompting methods with *gpt-3.5-turbo* model, retaining only error-inducing samples. In the filtering phase, we focus on code with compile errors and use the following processing and filtering criteria: extraction of code from markdown blocks, validation of module statements, and removal of samples with extraneous language or empty module bodies. The final step involved clustering using DBSCAN [16] with Jaccard distance [12], grouping similar implementations to select representative examples while ensuring a diverse representation of syntax errors. This results in a total of 212 erroneous implementations in the dataset.

4 EXPERIMENTS

In this section, we first present the evaluation metrics in Section 4.1, followed by our primary findings showcased in Section 4.2. Within this section, we show the performance improvements and the impact of ReAct and RAG. Finally, Section 4.3 details ablation studies on the quality feedback message and LLM.

Setup: We conduct all experiments with GPT-3.5 as the LLM through OpenAI APIs [13], except for the ablation experiment on different LLM. We specifically used *gpt-3.5-turbo-16k-0613*. A simple rule-based syntax fixer is applied to every LLM-generated verilog code, which avoids simple errors such as misplaced timescale derivatives. In all experiments, we set the sampling temperature to 0.4. For ReAct prompting, we restrict the LLM to a maximum of 10 iterations of Thought-Action-Observation, where Action might involve interactions with the compiler. We consider the syntax error resolved if any of the generated code passes. To limit test variance, we repeat each experiment 10 times and report the average.

4.1 Evaluation Metric

Compile Fix rate: To demonstrate the debugging capability of our method, we calculate the expectation fix rate, with c as the number of fixed samples out of all $n = 10$ samples.

$$\text{fix rate} = \mathop{\mathbb{E}}_{problems}\left[\frac{c}{n}\right] \tag{1}$$

Functional Correctness: We follow recent work in directly measuring code functional correctness with simulation through pass@k metric [2], where a problem is considered solved if any of the k samples passes the tests. We use the unbiased estimator as follow and ensure $n = 20$ is sufficiently large:

$$\text{pass@k} = \mathop{\mathbb{E}}_{problems}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right] \tag{2}$$

4.2 Main Results

Prompt	RAG	Simple	iverilog	Quartus	GPT-4
One-shot	w/o	0.414	0.536	0.587	0.91
	w/	-	0.800	0.899	0.98
ReAct	w/o	0.671	0.731	0.799	0.92
	w/	-	0.820	0.985	0.99

Table 1: Fix rate for One-shot vs. ReAct, w/ and w/o RAG, ablation on feedback quality and LLMs on VerilogEval-syntax.

Dataset VerilogEval	Set	pass@1 original	pass@1 fixed	pass@5 original	pass@5 fixed
Human	All	0.267	0.368	0.458	0.506
	easy	0.521	0.666	0.808	0.847
	hard	0.053	0.120	0.164	0.221
Machine	All	0.467	0.799	0.691	0.891
	easy	0.568	0.833	0.782	0.892
	hard	0.367	0.771	0.601	0.890

Table 2: Pass@k for simulation pass rate on VerilogEval dataset after fixing syntax errors.

The main results in Table 1 show the effectiveness of ReAct and RAG which each provided performance gain in a large margin. We note that One-shot generation includes only a single-turn interaction with either simple or compiler message as feedback. Table 2 showed the improvement of pass@{1,5} on VerilogEval dataset after fixing syntax errors with visualizations in Figure 4. The VerilogEval-Human benchmark statistics show that syntax errors constitute a significant 55% of errors in GPT-3.5 generated Verilog code, surpassing simulation errors. With just addressing syntax errors using our approach (shown in the inner circle), the pass rate increases from 26.7% to 36.8%. Table 3 shows that our approach can generalize across different benchmarks. We further detail our findings below.

Impact of ReAct: ReAct significantly outperforms One-shot generation. ReAct's ability to iteratively revise code with reasoning and planning results in superior performance. Moreover, even without explicit feedback from the compiler (Simple), the intermediate reasoning steps similar to chain-of-thought can still bring considerable

improvements from 41.4% to 67.1%. When compared with One-shot without RAG, ReAct enhances syntax success rates by 25.7%, 26.4%, and 31.2% with iterative feedback from Simple, iverilog, and Quartus, respectively. We also observe consistent improvement from ReAct, regardless of the compiler and use of RAG.

Impact of RAG: Notably, the application of RAG with human expert guidance boosts the fix rate considerably and substantially enhances the solution's reliability. The results with Quartus compiler in Table 1 show that RAG improves the fix rate by 31.2% (58.7% to 89.9%) for One-shot and 18.6% (79.9% to 98.5%) when using ReAct. We observe consistent improvement with RAG, regardless of the quality of the compiler feedback message and LLM (GPT-4).

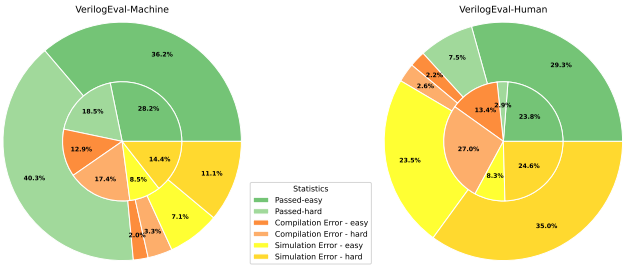


Figure 4: VerilogEval pass@1 results prior (inner) and post (outer) syntax error fixing with RTLFixer.

LLM	Syntax Success Rate	pass@1
GPT-3.5	73%	11%
GPT-3.5 + **RTLFixer**	93%	16%

Table 3: Improvements of Syntax Success rate and simulation Pass@1 on RTLLM benchmark using ReAct and RAG with Quartus compiler.

Simulation Correctness Improvement: Previous research [9, 11] evaluates the performance of LLM-generated Verilog code using the pass@k metric. However, this approach doesn't account for syntax errors in the code samples, which can skew accuracy. In our study, we evaluate functional correctness using the VerilogEval benchmark, specifically addressing fixes to syntax errors in the code samples. The results, displayed in Table 2, show the performance scores on the VerilogEval dataset and the improvements after rectifying syntax errors, with 32.3% and 10.1% improvement on the pass@1 metric for Machine and Human respectively. We further divided the VerilogEval benchmark into two subsets: *easy*, comprising 71 problems, and *hard*, consisting of 85 problems. These subsets have been delineated based on a pass rate threshold of 0.1 on Human. For simple problems in Human and low-level descriptions in the Machine, the correction of syntax errors significantly enhances the pass rate, reaching around 80% for pass@1. When contrasting the pass rate improvements between easy and hard problems in the Human descriptions, we observe a greater improvement for easy problems at 14.5% compared to hard problems at 6.7% for pass@1. This discrepancy suggests that LLMs still face challenges when advanced reasoning and problem-solving skills are required. Discussions on future work to address simulation errors are presented in Section 5.

Generalizability: Our method using ReAct and RAG can be generalized to other benchmark. To account for potential overfitting during the design of the retrieval database, we also tested our method on the RTLLM [11] benchmark without deriving new human guidance for the retrieval database. As shown in Table 3, our framework improves the syntax success rate from 73% to 93%, demonstrating its capability to generalize[1].

4.3 Ablation Studies

Our research includes two ablation studies designed to evaluate the impact of feedback quality and the selection of LLMs on the effectiveness of syntax error correction.

4.3.1 Impact of Feedback Quality. We study the impact of feedback quality with using different feedback messages detailed below.

Simple: We only give an instruct prompt "*Correct the syntax error in the code.*" without any explicit instruction on what the error is about and how to fix it.

Icarus Verilog (iverilog) [19]: Open-source Verilog simulator. The compiler occasionally encounters edge cases where it fails to provide informative logs, outputting messages such as "*I give up.*" Logs lack clarity, making them challenging to decipher.

Quartus[2]: Commercial compiler for FPGAs. In contrast with the open-source counterpart, it delivers well-defined and clear logs, effectively identifying errors and often offering suggestions and validation tips, making it more user-friendly and informative.

We deem Simple, iverilog, and Quartus to have increasing level of feedback message quality and illustrate the difference of the two compilers with an example in Figure 5. Results depicted in Table 1 clearly demonstrate that compiler logs give better feedback than Simple feedback and that the quality of compiler output impacts the performance of LLM debugging. As the quality of the compiler message improves (iverilog v.s. Quartus), the success rate of fixing syntax errors also increases. Intriguingly, the disparity between iverilog and Quartus results is more pronounced when using React with RAG. This discrepancy potentially suggests that high-quality compiler messages enhance the LLM's ability to more effectively utilize the retrieved human expert guidance.

4.3.2 Impact of Different LLMs. In Table 1, we present the results obtained when utilizing GPT-4 as the underlying LLM with Quartus as the compiler. A notable improvement in syntax error resolution is observed when using GPT-4 compared to GPT-3.5, particularly with One-shot prompting with RAG, where the success rate increased from 89.9% to 98%. When comparing the results of GPT-4 between One-shot and ReAct, we observe minor improvements of approximately 1%, suggesting that GPT-4 is already a robust agent proficient in fixing syntax errors without the need for reasoning, action planning, and iterative refinement.

Nonetheless, it is important to highlight that our approach of empowering LLMs with ReAct and RAG can significantly narrow the gap between weaker LLMs and stronger ones, especially beneficial for weaker open-source models [9, 17] that may not be as performant as GPT-4 on programming tasks.

[1]The syntax success rate we collected is different from the original paper because we used the Verilog code sample provided in the their repo for each problem.
[2]https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime/resource.html

Task ID: vector100r

Erroneous Implementation

```
module top_module (
input [99:0] in,
output reg [99:0] out
);
always @(posedge clk) begin
  for (int i = 0; i < 100; i = i +1) begin
    out[i] <= in[99 - i];
  end
end
endmodule
```

iverilog:
vector100r.sv:5: error: Unable to bind wire/reg/memory 'clk' in 'top_module'
vector100r.sv:5: error: Failed to evaluate event expression 'posedge clk'.
2 error(s) during elaboration.

Quartus:
Error (10161): Verilog HDL error at vector100r.sv(5): object "clk" is not declared. Verify the object name is correct. If the name is correct, declare the object. File: /tmp/tmp4u6ib9ig/vector100r.sv Line: 5 Error: Quartus Prime Analysis & Synthesis was unsuccessful. 1 error, 1 warning

Figure 5: Example of compiler log from iverilog and Quartus. Quartus feedback messages are more informative.

5 ANALYSIS AND DISCUSSION

In this section, we delve into a series of analyses and discussions, extracting valuable insights from our discoveries. Specifically, we provide analysis in fail cases and the effect of iterative code refinement. Additionally, we discuss the challenges associated with applying our method to debugging simulation logic errors.

Failure due to LLM's Incapability: Most of the cases where, even with the aid of ReAct and RAG, failed to correct syntax errors is due to the fundamental incapability of the LLM. Figure 6 illustrates one of the failure case where it particularly requires arithmetic index calculations to solve the index out-of-range error. Some other notable failures occurred in cases where LLMs were confident in incorrect syntax, possibly due to it being accepted in C/C++.

Erroneous Implementation (Partial):

```
for (i = 0; i < 16; i = i + 1) begin : ROW
    for (j = 0; j < 16; j = j + 1) begin : COLUMN
        neighbors[0] = q[(i-1)*16 + (j-1)];
        row_above = q[((i-1) & 15)*16 + j];
        ...
```

Compile Error:
Error (10232): Verilog HDL error at conwaylife.sv(23): index -17 cannot fall outside the declared range [255:0] for vector "q"

Figure 6: An example which the agent failed to fix a syntax error. LLM failed to calculate array indices in the for loop and does not recognize the out-of-bound error.

Iterative Code Refinement: In Figure 7, we analyze the number of iterations ReAct requires to fix syntax errors. About 90% of problems are resolved in a single revision. For the remaining cases, additional code revisions are necessary, as new errors may surface after addressing the initial ones.

Challenges in Debugging Simulation Errors: While our framework could be readily adapted to employ LLMs for debugging simulation errors, our preliminary studies revealed limited improvements beyond syntax error fixes. Despite our efforts to provide simulation error logs as feedback to LLM agents, including summaries on output error count and text-formatted waveform-like comparisons of error versus solution output, we observed that LLMs

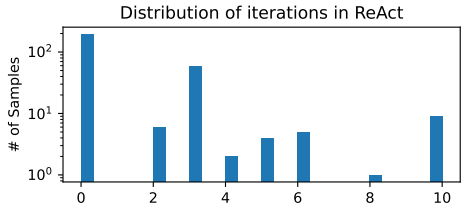


Figure 7: Distribution of iterations required by ReAct to fix syntax errors.

had constrained capabilities to comprehend simulation feedback messages. They only exhibited proficiency in fixing logic implementation errors for simple problems but struggled with more complex questions, especially those involving high-level design functionality descriptions and advanced reasoning. Addressing the challenges associated with LLMs fixing erroneous implementations in such problems, particularly those requiring advanced reasoning and problem-solving skills, remains an exciting area for future research. This also highlights the need to improve LLM's capabilities in reasoning and problem-solving related to hardware design.

6 CONCLUSION

Our framework **RTLFixer** demonstrates the significant impact of employing Retrieval Augmented Generation (RAG) and advanced prompting methods like ReAct in debugging Verilog code with Large Language Models. Key findings indicate that these approaches notably enhance syntax error resolution, achieving success rates as high as 98.5%. This research not only offers a novel autonomous language agent for Verilog code debugging but also introduces comprehensive dataset for further exploration.

REFERENCES

[1] Jason Blocklove, et al. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *arXiv preprint arXiv:2305.13243* (2023).
[2] Mark Chen, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
[3] Xinyun Chen, et al. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).
[4] Nat Friedman. 2021. Introducing GitHub Copilot: your AI pair programmer. (2021). https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer
[5] Ziwei Ji, et al. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (March 2023), 1–38. https://doi.org/10.1145/3571730
[6] Shuyang Jiang, et al. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *arXiv preprint arXiv:2306.02907* (2023).
[7] Patrick Lewis, et al. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:cs.CL/2005.11401
[8] Mingjie Liu, et al. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. arXiv:cs.CL/2311.00176
[9] Mingjie Liu, et al. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. *arXiv preprint arXiv:2309.07544* (2023).
[10] Shuai Lu, et al. 2022. ReACC: A Retrieval-Augmented Code Completion Framework. arXiv:cs.SE/2203.07722
[11] Yao Lu, et al. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).
[12] Suphakit Niwattanakul, et al. 2013. Using of Jaccard coefficient for keywords similarity. In *Proceedings of the international multiconference of engineers and computer scientists*, Vol. 1. 380–384.
[13] OpenAI. 2023. OpenAI models api. (2023). https://platform.openai.com/docs/models
[14] Hammond Pearce, et al. 2020. Dave: Deriving automatically verilog from english. In *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*. 27–32.
[15] Yujia Qin, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).
[16] Erich Schubert, et al. 2017. DBSCAN revisited, revisited: why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems (TODS)* 42, 3 (2017), 1–21.
[17] Shailja Thakur, et al. 2023. VeriGen: A Large Language Model for Verilog Code Generation. *arXiv preprint arXiv:2308.00708* (2023).
[18] Jason Wei, et al. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:cs.CL/2201.11903
[19] Stephen Williams et al. 2002. Icarus verilog: open-source verilog more than a year later. *Linux Journal* 2002, 99 (2002), 3.
[20] Shunyu Yao, et al. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
[21] Fengji Zhang, et al. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570* (2023).

TITAN: A Distributed Large-Scale Trapped-Ion NISQ Computer

Cheng Chu[†] Zhenxiao Fu[†] Yilun Xu[‡] Gang Huang[‡] Hausi Muller[*] Fan Chen[†] Lei Jiang[†]
[†]Indiana University [‡]Lawrence Berkeley National Laboratory [*] University of Victoria
[†]{chu6, zhfu, fc7 jiang60}@iu.edu [‡]{yilunxu, ghuang}@lbl.gov [*]hausi@uvic.ca

ABSTRACT

Trapped-Ion (TI) technology offers potential breakthroughs for Noisy Intermediate Scale Quantum (NISQ) computing. TI qubits offer extended coherence times and high gate fidelity, making them appealing for large-scale NISQ computers. Constructing such computers demands a distributed architecture connecting Quantum Charge Coupled Devices (QCCDs) via quantum matter-links and photonic switches. However, current distributed TI NISQ computers face hardware and system challenges. Entangling qubits across a photonic switch introduces significant latency, while existing compilers generate suboptimal mappings due to their unawareness of the interconnection topology. In this paper, we introduce TITAN, a large-scale distributed TI NISQ computer, which employs an innovative photonic interconnection design to reduce entanglement latency and an advanced partitioning and mapping algorithm to optimize matter-link communications. Our evaluations show that TITAN greatly enhances quantum application performance by 56.6% and fidelity by 19.7% compared to existing systems.

CCS CONCEPTS

• **Computer systems organization → Distributed architectures**; • **Hardware → Quantum technologies**.

KEYWORDS

Trapped Ions, NISQ, Distributed Quantum Computers

ACM Reference Format:
Cheng Chu, Zhenxiao Fu, Yilun Xu, Gang Huang, Hausi Muller, Fan Chen, and Lei Jiang. 2024. TITAN: A Distributed Large-Scale Trapped-Ion NISQ Computer. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3655908

1 INTRODUCTION

TI technology emerges as a promising avenue for the construction of large-scale NISQ computers, potentially unlocking quantum advantages [7]. TI qubits present several distinct advantages: longer coherence times, up to one hour [21] compared to conventional superconducting qubits; higher gate fidelity, with 2-qubit gates achieving 99.92% fidelity [9]; dense qubit connectivity for efficient 2-qubit gate implementation [12]; and modular, scalable QCCDs [17], exemplified by the latest Quantinuum's 32-qubit QCCD [17].

To construct large-scale TI NISQ computers, a distributed architecture is essential. Integrating numerous qubits within a single QCCD device significantly degrades TI gate fidelity [18]. While quantum matter-links [1] can connect QCCDs into a TI module, excessive QCCD integration leads to cross-talk between QCCDs [16] and increased control and cooling overhead [6]. Therefore, large-scale TI NISQ computers connect distributed QCCD-based TI modules via a photonic switch [15].

However, state-of-the-art distributed TI NISQ computers face challenges from both hardware and system perspectives. On the hardware side, entangling two qubits across a photonic switch requires a significant latency, $\sim 60\times$ longer than a 2-qubit gate [15], impacting quantum application performance. The entanglement process involves establishing optical connections, cooling, and ~ 10 entanglement attempts, each lasting $500\mu s$ [20]. While more switch ports can reduce the final step's latency, they prolong optical connection establishment time. Thus, simply adding ports does not reduce overall entanglement latency. On the system side, state-of-the-art compilers [3, 23] designed for distributed quantum computers generate unoptimized qubit mappings, due to their lack of awareness regarding the interconnection. These compilers minimize inter-module communications by graph partitioning, but overlook the presence of quantum matter-links, as well as the specific locations of photonic ports within each TI module. Consequently, the mappings generated by these compilers inadvertently lead to frequent communications across quantum matter-links.

In this paper, we propose *TITAN*, a large-scale distributed TI NISQ computer, where multiple QCCDs are interconnected as a TI module by quantum matter-links, and multiple distributed TI modules are interconnected using a photonic switch. Our contributions are summarized as follows.

- **Innovative Photonic Interconnection Design**. We introduce an innovative photonic interconnection design for TITAN to accelerate entanglements across a photonic switch. By increasing the number of ports within each QCCD-based TI module, TITAN enhances the concurrency of entanglement attempts, thus reducing the latency associated with entangling qubits across a photonic switch. Instead of using a single large, slow photonic switch, TITAN employs multiple smaller, faster photonic switches to connect TI modules.
- **Advanced Partitioning and Mapping Algorithm**. We present a novel partitioning and mapping algorithm to curtail communication overhead across quantum matter-links. Our algorithm minimizes inter-module communications and inter-QCCD communications within each TI module through hierarchical partitioning. Furthermore, the algorithm optimizes communication patterns across quantum matter-links by allocating the qubit partition characterized by the highest frequency of inter-module

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3655908

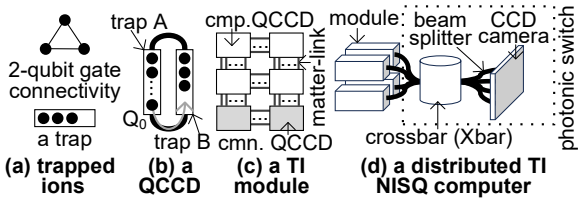


Figure 1: The basics of TI technology, a trap, a QCCD, a module, and a distributed architecture.

communications to the QCCD positioned nearest to the photonic port within a TI module.

- **Enhanced Performance and Fidelity**. We evaluated and compared TITAN against existing distributed TI NISQ computers with previous compiler support. Our assessments demonstrate that, compared to previous TI NISQ computers, TITAN improves the performance and fidelity of various quantum applications by 56.6% and 19.7%, respectively.

2 BACKGROUND AND MOTIVATION

2.1 Trapped-Ion Technology

Ion Trap and Gate. In Trapped-Ion (TI) quantum systems, information is encoded within ions confined within an ion trap [21]. Electrode segments at each end of the trap create a spatial confinement, while a radio-frequency electric field induces fluctuations, arranging ions into a linear chain, as shown in Figure 1(a). Quantum gates are realized through laser manipulation. Single-qubit gates involve interactions with specific ions, while 2-qubit gates require multiple lasers to excite both internal states and ion chain motion, enabling full qubit connectivity. The Mølmer-Sørensen (MS) gate the canonical 2-qubit gate, known for lower fidelity compared to single-qubit gates in TI systems [9].

Quantum Charge Coupled Device. The fidelity of qubits is compromised when controlling and implementing quantum gates in a long ion chain [18], making a single-trap architecture unsuitable for scalability. To address this issue, a modular and scalable Quantum Charge Coupled Device (QCCD) [17] is introduced for constructing large-scale TI NISQ computers, as illustrated in Figure 1(b). The QCCD consists of two ion traps, each with a small number of ions, interconnected by conveyor belt regions [17]. To physically move a qubit (Q_0) from one trap (A) to the other (B), a split operation separates Q_0 from the ion chain in A, followed by a shuttling operation to move Q_0 to B and a merge operation to combine it with the ion chain in B. Finally, a 2-qubit gate can occur between Q_0 and another qubit in B.

TI Module. A TI module comprises multiple QCCDs interconnected via quantum matter-links [1], as depicted in Figure 1(c). These quantum matter-links facilitate ion transfer between QCCDs through electric fields. At the QCCD boundary, an RF electrode aligns with the corresponding electrode of the adjacent QCCD. By applying translating potentials across the inter-QCCD gap, ions can be transported. The transportation of a qubit through a matter-link takes $0.4ms$ and attains a fidelity of 99.999993% [1]. To minimize cross-talk, specific QCCDs are configured with more $^{171}Yb^{+}$ ions for computing (cmp.), while others emphasize $^{138}Ba^{+}$ ions, optimized for communication (cmn.) tasks [5].

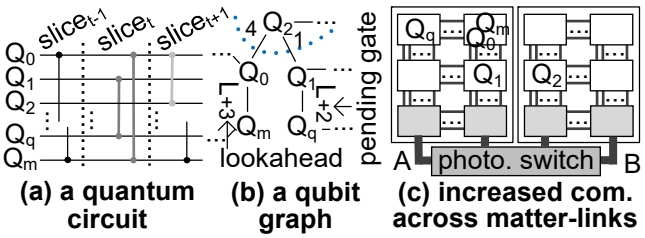


Figure 2: Prior compilers on a distributed TI NISQ computer.

2.2 Distributed TI NISQ Computer

Distributed TI NISQ Computer. A large-scale TI NISQ computer is constructed by interconnecting multiple distributed TI modules using a photonic switch, as shown in Figure 1(d). A photonic switch [16] comprises components such as a MEMS optical crossbar (Xbar), beam splitters, and a CCD camera detector array. It enables the heralded and probabilistic distribution of entanglement between two TI modules.

Entanglement via Photonic Switch. Entangling qubits across a photonic switch [20] involves three phases: optical connection establishment, a cooling operation, and multiple entanglement attempts. The process begins with the "Xbar operation", where the photonic switch's Xbar component establishes an optical connection between incoming and outgoing ports. Subsequently, the photonic switch proceeds to the "non-Xbar operations", which include a cooling operation lasting $\sim 100\mu s$, followed by multiple entanglement attempts, each taking $500\mu s$ [20]. These attempts are crucial for achieving a successful entanglement. Completing all non-Xbar operations for a single entanglement requires $\sim 5.4ms$ [5], involving about 10 attempts. Notably, an entanglement incurs a significant latency, $\sim 60\times$ longer than that of a TI 2-qubit gate, which adversely affects the performance of distributed TI NISQ computers.

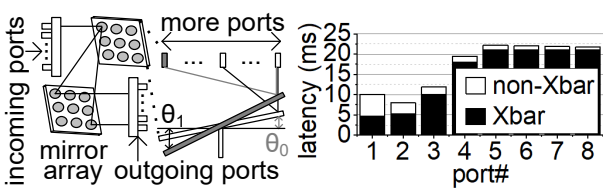
2.3 Compiler for Distributed NISQ Computer

Qubit Graph. A qubit graph [3, 23] plays a crucial role in optimizing mappings for quantum circuits in distributed NISQ computers. A circuit is divided into multiple time-slices, each containing a set of concurrently executed gates, as depicted in Figure 2(a). These time-slices are then abstracted into qubit graphs, with nodes representing data qubits and edges weighted to denote the number of 2-qubit gates, as highlighted in Figure 2(b).

Lookahead Weight. Relying solely on information from the current time-slice often results in suboptimal mappings. A *lookahead* mechanism [3, 23] is introduced to construct qubit graphs that span a more extended temporal range within the quantum circuit by enriching these qubit graphs with lookahead weights. The process of generating a qubit graph at time t begins with the original qubit graph in time slice t and assigns a super large weight ("L") to edges connecting interacting qubits in the current time slice, guaranteeing that any mapping strategy will position these qubits within the same partition. For each qubit pair, the weight [3, 23] of their edge is computed as follows:

$$w_t(Q_i, Q_j) = \sum_{t<m<T} I(m, Q_i, Q_j) \cdot D(m-t), \tag{1}$$

where D denotes an exponential decay function, i.e., $D(x) = 2^{-x/\sigma}$, $I(m, Q_i, Q_j)$ is an indicator variable (equal to 1 if Q_i and Q_j interact in time slice m; and 0 otherwise), and T represents the total number



(a) The Xbar operations in a larger Xbar. (b) The overall entanglement latency.

Figure 3: Adding more ports for larger entanglement attempt concurrency in an entanglement.

of time-slices in the circuit. This augmentation of qubit graphs with lookahead weights effectively takes into account the influence of upcoming time-slices, giving larger weights to interactions occurring sooner in the circuit.

Graph Partitioning and Mapping. Previous compilers [3, 23], designed for distributed quantum computing setups that encompass various NISQ computing units linked via a central hub, employ a recurring procedure involving the Kernighan-Lin algorithm. This process is geared towards the division of the qubit graph into several partitions of uniform size. Subsequently, each partition is indiscriminately assigned to one of the NISQ computing devices.

2.4 Motivation

Unfortunately, the performance and fidelity of state-of-the-art distributed TI NISQ computers are constrained by a combination of hardware- and system-level challenges.

Slow Photonic Switch. A key impediment leading to significant entanglement latency is found in the final phase, i.e., the non-Xbar operations, which comprises multiple entanglement attempts, each consuming 500μs [20]. Simply increasing the number of ports within the Xbar component of the photonic switch to enhance entanglement attempt concurrency does not effectively alleviate the overall entanglement latency. This is due to the fact that a larger Xbar, housing additional ports, substantially prolongs the duration required for establishing optical connections within the Xbar, i.e., the Xbar operation [13]. As Figure 3(a) illustrates, the Xbar necessitates a prolonged latency to rotate the mirror arrays by a larger angle ($\theta_1 > \theta_0$) to accommodate an increased number of incoming and outgoing ports [13]. While Figure 3(b) suggests that the latency of non-Xbar operations diminishes with an increasing number of Xbar ports, this advantage is offset by the protracted duration of the Xbar operation induced by the enlarged Xbar.

Compiler Limitations in Interconnection Handling. Previous compilers [3, 23], designed for general distributed NISQ computers, while adept at minimizing communications between different TI modules, inadvertently amplify qubit movements across quantum matter-links, due to their lack of awareness regarding the interconnection of a distributed TI NISQ computer. As shown in Figure 2(b), these compilers effectively partition the qubit graph into two segments and subsequently map each segment to a TI module, achieving great reductions in inter-module communications. However, Figure 2(c) illustrates a potential pitfall. In some instances, these compilers may allocate two qubits, denoted as Q_1 and Q_q, recognized for their frequent inter-QCCD communications within module A, to two separate QCCDs that lack direct connectivity. This mapping choice results in an increased volume of ion movements across matter-links. The underlying cause of this issue lies in the absence of support for QCCD-level partitioning and mapping within each TI module by these previous compilers. Furthermore, another scenario arises where these compilers may allocate two qubits, Q_1 and Q_2, which are characterized by frequent inter-module communications, to two computing QCCDs positioned in separate TI modules without direct connections to their respective photonic ports. Consequently, this mapping strategy inevitably introduces additional communication loads across matter-links between a computing QCCD and a communication QCCD within each TI module.

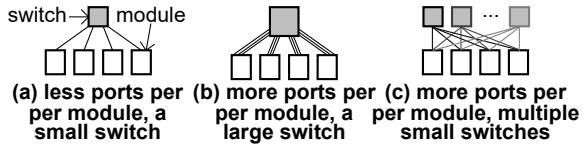


Figure 4: The photonic interconnection in TITAN.

3 TITAN

In this paper, we present *TITAN*, a rapid distributed TI NISQ computer that interconnects multiple QCCDs as a TI module using quantum matter-links, and further links multiple distributed TI modules through a photonic switch. TITAN has two innovative features: a novel photonic interconnection design and an advanced partitioning and mapping algorithm. Firstly, TITAN revolutionizes photonic interconnections to significantly expedite entanglement processes across the photonic switch. By augmenting the number of ports within each TI module, TITAN greatly improves the concurrency of entanglement attempts. This enhancement effectively reduces the latency associated with qubit entanglement through the photonic switch. Unlike conventional approaches employing single large and slow photonic switches, TITAN adopts a more efficient strategy with multiple smaller, faster photonic switches for interconnecting TI modules. Secondly, TITAN incorporates an innovative algorithm for partitioning and mapping. Our algorithm minimizes inter-module communications and inter-QCCD communications within each TI module through hierarchical partitioning. And it also optimizes communication patterns across matter-links by mapping the qubit partition with the highest frequency of inter-module communications to the communication QCCD situated nearest to the photonic port within a TI module. Our partitioning and mapping algorithm ensures a more optimized distribution of qubits, leading to a substantial reduction in communications that traverse quantum matter-links.

3.1 Innovative Photonic Interconnection Design

More Ports in a TI Module. TITAN leverages a photonic switch [16] to interconnect multiple distributed TI modules. To establish an entanglement, the photonic switch activates its optical Xbar, enabling optical connections between two TI modules. The time needed to configure these connections is referred to as the Xbar latency. Subsequently, the source TI module undergoes ion cooling ($\sim 100\mu s$) and executes around $\sim$ 10 entanglement attempts, each consuming 500μs [20], to successfully complete the entanglement with the destination TI module across the photonic switch. This duration, encompassing cooling and multiple entanglement attempts, is known as the non-Xbar latency. In previous distributed TI NISQ computers [16], each TI module was equipped with only a limited number of ports, and a small, high-speed photonic switch was utilized to interconnect them, as depicted in Figure 4(a). However, the restricted

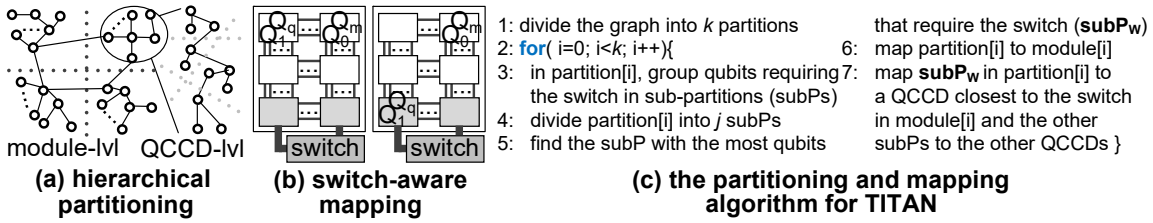


Figure 5: The compiler design for TITAN comprising distributed TI modules interconnected by a photonic switch.

number of ports in a TI module severely curtails the concurrency of entanglement attempts in an entanglement, substantially extending the non-Xbar latency. As illustrated in Figure 4(b), we propose a significant augmentation of the port count within each TI module in TITAN to amplify entanglement attempt concurrency and, consequently, reduce the non-Xbar latency. Nevertheless, the integration of more ports into a TI module significantly enlarges the Xbar size of the photonic switch, consequently slowing down the Xbar latency of an entanglement. Therefore, the mere addition of ports to a TI module is insufficient to reduce the overall latency of an entanglement, as highlighted in Figure 3(b).

Multiple Compact Photonic Switches. To diminish both Xbar and non-Xbar latencies within an entanglement, TITAN adopts multiple smaller photonic switches to connect its TI modules, as illustrated in Figure 4(c). Suppose a prior TI distributed NISQ computer comprises n TI modules, each equipped with m ports (where $n \geq 2$ and $m \geq 1$), the interconnection necessitates a large $nm \times nm$ photonic switch. TITAN, on the other hand, augments the number of ports within each TI module by a factor of 10×, thereby endowing each TI module with $10m$ ports. Instead of using a single $10nm \times 10nm$ photonic switch, TITAN adopts $10m$ individual $n \times n$ photonic switches to connect the TI modules. This approach ensures that each port of a TI module is connected to a distinct photonic switch, as it is impossible for the ports of the same TI module to communicate with each other. The $n \times n$ photonic switch, responsible for interconnecting the many-ported TI modules within TITAN, exhibits a more compact form, thereby rendering it swifter compared to the $nm \times nm$ photonic switch employed by the earlier distributed TI NISQ computer.

3.2 Partitioning and Mapping

The pseudocode of our partitioning and mapping algorithm is described in Figure 5(c).

Hierarchical Partitioning. Previous compilers for distributed quantum computers [3, 23] employ the Kernighan-Lin algorithm to partition a qubit graph into multiple equally-sized partitions, each subsequently mapped to a dedicated NISQ computer. However, this approach cannot be straightforwardly applied to partition a qubit graph for a distributed TI NISQ computer, which comprises k TI modules, each housing j QCCDs. One naive approach might involve dividing the qubit graph into $k \cdot j$ partitions of equal size and mapping each to a designated QCCD. However, this naive solution would require invoking the Kernighan-Lin algorithm for $q \cdot C^2_{j \cdot k}$ times, where q is the number of optimization iterations. The Kernighan-Lin algorithm has a time complexity of $O(pn^2 \log n)$ [3], where n is the number of qubits, and p represents the number of optimization iterations. Consequently, the time complexity of this naive solution scales as $O(q \cdot C^2_{j \cdot k} pn^2 \log n)$. When k, j, and n are sufficiently large, this approach becomes impractically slow. In contrast, as Figure 5(a) shows, we propose a *hierarchical partitioning* technique to efficiently divide the qubit graph into $k \cdot j$ partitions. Similar to previous compilers, we initially partition the qubit graph into k partitions of equal size using the Kernighan-Lin algorithm (Line 1 of Figure 5(c)). Each partition is then mapped to a specific TI module (Line 6). Subsequently, we further divide each partition into j smaller sub-partitions of equal size (Line 4), with each sub-partition being assigned to a QCCD within the corresponding TI module (Line 7). The foundation of our hierarchical partitioning lies in the recognition that the latency of the photonic switch connecting TI modules exceeds that of the quantum matter-links interconnecting QCCDs within each TI module. This hierarchical partitioning approach maintains a time complexity of $O((kpn^2 \log n + jp(n/k)^2 \log(n/k))$, much smaller than the naïve solution. The goal of our hierarchical partitioning is to group two logic qubits frequently interacting with each other in a sub-partition. For instance, unlike the inefficient partitions made by previous compilers and shown in Figure 2(c), as Figure 5(b) shows, our hierarchical partitioning can group Q_1 and Q_q sharing a large weight in one sub-partition, and Q_0 and Q_m joining a pending 2-qubit gate in another sub-partition to reduce the inter-QCCD communications within each TI module.

Switch-aware Mapping. Previous compilers tailored for distributed NISQ computers [3, 23] have a uniform mapping approach where each partition is assigned to a distributed NISQ computer without considering any specific attributes. This approach is suitable when dealing with the mapping of large partitions of the qubit graph to TI modules. However, directly applying this uniform approach when mapping smaller sub-partitions to individual QCCDs results in suboptimal mappings. This is due to variations in the distances between each QCCD and the photonic ports within a TI module. To address this issue, we propose a *switch-aware* mapping scheme that optimizes the placement of sub-partitions within a TI module to minimize ion movements. Our switch-aware mapping approach takes into account the unique characteristics of each TI module's interconnection setup. In our switch-aware mapping, after the module-level separation achieved through hierarchical partitioning, instead of randomly initializing all sub-partitions, we first group qubits that communicate frequently with other partitions into several sub-partitions (Line 3 of Figure 5(c)), while the remaining sub-partitions are initialized randomly. After the hierarchical partitioning, our switch-aware mapping strategy identifies the sub-partition with the highest frequency of communication with other partitions within each partition (Line 5). It then assigns this sub-partition to the communication QCCDs that are closest to the photonic ports within the TI module (Line 7). The other sub-partitions are mapped to the remaining QCCDs in the TI module. As

Table 1: The design overhead comparison.

Scheme	Description
baseline	4 TI modules are interconnected by a 256 × 256 photonic switch. A module has 64 photonic ports and consists of 6 QCCDs, each having up to 32 qubits. 8 matter-links connect two neighboring QCCDs in a TI module. For an entanglement across the switch, Xbar - 5.23ms & non-Xbar - 2.75ms. 512 physical qubits: 256 data qubits & 256 for com.
TITAN	TI module and QCCD configurations are the same as baseline. TITAN employs 8 32 × 32 switches to interconnect 4 TI modules. For an entanglement across a switch, Xbar - 1.1ms & non-Xbar - 0.765ms.

Table 2: The simulated benchmarks

benchmark	logic qubit	2-qubit gate #
Adder (ADD)	256	2033
Bernstein–Vazirani (BV)	256	255
QAOA (QAO)	256	1020
Quantum Primacy (PRI)	256	192
Random (RAN)	256	2705
Hamiltonian (HAM)	256	510

illustrated in Figure 5(b), for instance, the sub-partition comprising qubits Q_0 and Q_m exhibits the most frequent communication with other partitions. Our switch-aware mapping approach, rather than assigning this sub-partition to a QCCD in the first row of the TI module, places it in one of the communication QCCD closest to the photonic ports, which, in this case, is the last row of the TI module.

4 EXPERIMENTAL METHODOLOGY

Baseline Configuration. The hardware configuration of our baseline is shown in Table 1. We have adopted the most recent racetrack QCCD design [17], accommodating up to 32 qubits. Six QCCDs are interconnected as a TI module, with four QCCDs dedicated to computing tasks and the remaining two tailored for communication functions. Each QCCD employs 8 quantum matter-links [1] to establish connections with neighboring QCCDs. The transportation of ions through a matter-link takes 0.4ms, attaining a fidelity of 99.999993% [1]. Within each TI module, there are 64 photonic ports. Based on Figure 3(b), an entanglement simultaneously performs two entanglement attempts to obtain the minimal overall latency. A module supports two concurrent entanglements. Moreover, we assume three distillation iterations for each entanglement, elevating its fidelity from an initial 94% [20] to a target of 99.3%, incurring an 8-qubit/port overhead. These four TI modules are interconnected via a 256 × 256 photonic switch. For entanglement operations across the switch, the Xbar latency is 5.23ms [13], while non-Xbar operations require 2.75ms [20]. Our baseline is composed of 512 physical qubits distributed across six QCCDs, with 256 designated for data storage and processing and the remaining 256 allocated to entanglements and their distillations. To compile quantum applications within our baseline, we employ a state-of-the-art compiler [3] specifically designed for general distributed NISQ computers. However, it is important to note that this compiler primarily supports module-level partitioning and mapping. Consequently, our baseline directly maps qubits within a partition to the four QCCDs within a TI module, based on their natural order.

Design Overhead. The design overhead is presented in Table 1. TITAN shares the same hardware configurations as our baseline, except the follows. Although a module also owns 64 photonic ports, it supports an entanglement by 8 concurrent entanglement attempts. TITAN employs 8 32 × 32 photonic switches to interconnect its four TI modules. Compared to the 256 × 256 switch, the 8 32 × 32 switches of TITAN reduce the size of mirror arrays by 87.5%. For an entanglement across a switch, the Xbar latency is 1.1ms [13], and the non-Xbar requires 0.765ms. Compared to the compiler [3] of our baseline, our TITAN's compiler has to perform QCCD-level partitioning and mapping having a time complexity of $O((jp(n/k)^2 \log(n/k))$, where n is the number of qubits, p represents the number of optimization iterations, j is the number of QCCDs in a TI module, and k is the number of TI modules.

Table 3: The timing and fidelity models.

operation	time (μs)	infidelity	operation	time (μs)	infidelity
1-qubit gate	5 [10]	3e-5	2-qubit gate	100 [18]	8e-4
merge/split	380 [10]	-	1-step shuttling	5 [18]	1e-5
X-Junction	100 [4]	1e-4	measurement	400 [10]	9e-5
matter-link	400 [1]	7e-8	photonic switch	5760	7e-3

Quantum Applications. Our study focuses on a range of quantum applications detailed in Table 2. In the context of our baseline, we specifically consider applications characterized by 256 data qubits and varying numbers of 2-qubit MS gates, spanning from 192 to 2.7K. *Adder* (ADD) [18] is frequently used in QFT and quantum phase estimation. *Bernstein–Vazirani* (BV) [22] determines an unknown bit string within a black-box function. *QAOA* (QAO) [14] addresses combinatorial optimization problems across diverse domains. *Quantum Primacy* (PRI) [2] generates random circuits to showcase quantum advantage. *Random* (RAN) [11] assembles quantum gates randomly, forming circuits for quantum machine learning. *Hamiltonian* (HAM) [19] generates circuits for simulating 1D Transverse Field Ising Models.

Timing & Fidelity. Our timing and fidelity models (Table 3) rely on latency and fidelity values from [18] for 2-qubit gates and shuttling, and from [10] for 1-qubit gates, merge/split, and measurement. A typical merge/split operation takes 80μs[10] but may reduce gate fidelity due to ion chain heating. To mitigate this, cooling operations during merging/splitting require ∼ 300μs[8]. The X-Junction design [4] and matter-link design [1] are applied. Entanglement latency across a photonic switch is calculated in Table 1.

Simulation. We modified and extended a state-of-the-art simulator [18] designed for a single QCCD to model multi-QCCD TI modules and distributed TI NISQ computers. Our modified simulator utilizes IBM's Qiskit framework for circuit processing and benchmark implementation.

5 EVALUATION

Performance. The performance of various quantum applications achieved by TITAN is shown in Figure 6. In our baseline (BASE), entanglements through the photonic switch constitutes an average of 66.2% of the total application latency, due to the long latency of the large switch. On average, our new photonic interconnection design (SWITCH) featured by small, low-latency Xbars yields a significant performance improvement of 48.6%. Through both hierarchical partitioning and switch-aware mapping of TITAN, on average, the application performance is furhter increased by 13.1% over SWITCH, due to the reduction of matter-link usage between QCCDs in each module. Meanwhile, the diminished ion movements across matter-links also decrease the frequency of various TI operations, including merge/split, shuttle, and X-junction. Overall, the amalgamation of both hardware and system techniques in TITAN results in a 56.6% increase in quantum application performance compared to BASE.

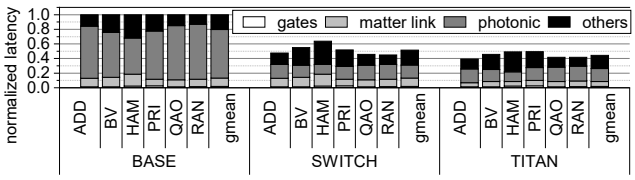


Figure 6: The quantum application performance of TITAN.

Fidelity. The fidelity of quantum applications of TITAN is exhibited in Figure 7. On average, SWITCH exhibits a 14.2% improvement in fidelity compared to BASE, due to its shorter application latency. The evolution of the state $|\psi\rangle$ of a quantum system can be described as $i\hbar\frac{d|\psi\rangle}{dt} = H|\psi\rangle$, where H is the Hamiltonian determining the evolution, $\hbar$ is the reduced Planck constant, and i is the imaginary unit. A reduced application latency leads to less decoherence and relaxation of quantum states, thereby improving fidelity even when the operation type and number remain unchanged. TITAN further reduces the TI operation count and noises in quantum circuits by hierarchical partitioning and switch-aware mapping, resulting in a 4.8% fidelity enhancement across all benchmarks. Overall, TITAN achieves a fidelity improvement of 19.7%.

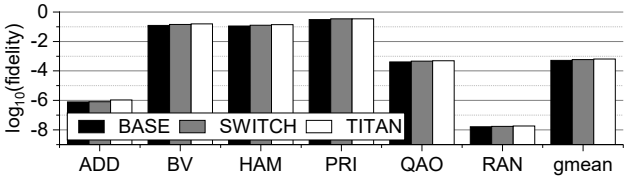


Figure 7: The quantum application fidelity of TITAN.

Partition & Mapping. Figure 8 illustrates the performance improvement achieved by hierarchical partitioning and switch-aware mapping of TITAN, with all results normalized to SWITCH. On average, hierarchical partitioning (HP) demonstrates a 8.5% performance improvement compared to SWITCH, since HP groups qubits frequently communicating with each other in a QCCD. The combination of HP and switch-aware mapping (SAM) achieves a 5.2% performance improvement over HP, since SAM maps qubits requiring entanglements to the communication QCCDs close to photonic ports. Overall, two schemes together decreases the number of ion movements across matter-links, leading to a 13.1% performance enhancement.

Sensitivity Study on Port #. Figure 9 depicts the performance variations across different port numbers in the photonic interconnection design of TITAN. All results are normalized to BASE with 64 ports. As the port number decreases, there is a corresponding increase in the latency of each benchmark. In comparison to BASE, the latencies for 48 ports, 32 ports, and 16 ports exhibit average increments of 13.46%, 23.67%, and 57.01%, respectively. This rise in latency is predominantly because of the reduced number of ports in each TI module, diminishing the concurrency of entanglement attempts and consequently leading to prolonged latency in every photonic entanglement.

6 CONCLUSION

In this paper, we present TITAN, a large-scale distributed TI NISQ computer featured by an innovative photonic interconnection design and an advanced partitioning and mapping algorithm. Our results show TITAN greatly enhances quantum application performance by 56.6% and fidelity by 19.7% compared to existing systems.

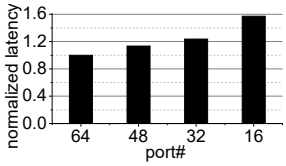


Figure 8: Study on Port #

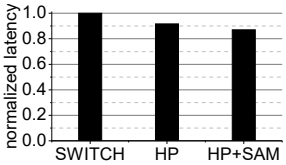


Figure 9: Partition & Mapping

ACKNOWLEDGMENTS

This work was supported in part by NSF CCF-2105972, and NSF CAREER AWARD CNS-2143120. We thank the IonQ Research Credits Program for their support.

REFERENCES

[1] M Akhtar et al. 2023. A high-fidelity quantum matter-link between ion-trap microchip modules. *Nature Communications* 14, 1 (2023), 531.
[2] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574, 7779 (2019), 505–510.
[3] Jonathan M. Baker et al. 2020. Time-Sliced Quantum Circuit Partitioning for Modular Architectures. In *ACM International Conference on Computing Frontiers*.
[4] R. B. Blakestad et al. 2009. High-Fidelity Transport of Trapped-Ion Qubits through an X-Junction Trap Array. *Physical Review Letters* 102 (2009), 153002. Issue 15.
[5] Kenneth R Brown et al. 2016. Co-designing a scalable quantum computer with trapped atomic ions. *npj Quantum Information* 2, 1 (2016), 1–10.
[6] Stefanie Castillo. 2023. The Electronic Control System of a Trapped-Ion Quantum Processor: a Systematic Literature Review. *IEEE Access* (2023).
[7] Andrew J Daley et al. 2022. Practical quantum advantage in quantum simulation. *Nature* 607, 7920 (2022), 667–676.
[8] L Feng et al. 2020. Efficient ground-state cooling of large trapped-ion chains with an electromagnetically-induced-transparency tripod scheme. *Physical Review Letters* 125, 5 (2020), 053001.
[9] J. P. Gaebler et al. 2016. High-Fidelity Universal Gate Set for $^{9}Be^{+}$ Ion Qubits. *Physical Review Letters* 117 (Aug 2016), 060505. Issue 6.
[10] M Gutiérrez et al. 2019. Transversality and lattice surgery: Exploring realistic routes toward coupled logical qubits with trapped-ion quantum processors. *Physical Review A* 99, 2 (2019), 022330.
[11] Chenyi Huang et al. 2023. Enhancing adversarial robustness of quantum neural networks by adding noise layers. *New Journal of Physics* 25, 8 (2023), 083019.
[12] David Kielpinski et al. 2002. Architecture for a large-scale ion-trap quantum computer. *Nature* 417, 6890 (2002), 709–711.
[13] William M. Mellette et al. 2015. Scaling Limits of MEMS Beam-Steering Switches for Data Center Networks. *Journal of Lightwave Technology* 33, 15 (2015).
[14] Nikolaj Moll et al. 2018. Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Science and Technology* 3, 3 (2018), 030503.
[15] Christopher Monroe et al. 2014. Large-scale modular quantum-computer architecture with atomic memory and photonic interconnects. *Physical Review A* 89, 2 (2014), 022317.
[16] Christopher Monroe and Jungsang Kim. 2013. Scaling the ion trap quantum processor. *Science* 339, 6124 (2013).
[17] S. A. Moses et al. 2023. A Race Track Trapped-Ion Quantum Processor. arXiv:2305.03828 [quant-ph]
[18] Prakash Murali et al. 2020. Architecting noisy intermediate-scale trapped ion quantum computers. In *IEEE International Symposium on Computer Architecture*.
[19] Lindsay Oftelie, , et al. 2020. Towards simulation of the dynamics of materials on quantum computers. *Physical Review B* 101, 18 (2020), 184305.
[20] LJ Stephenson et al. 2020. High-rate, high-fidelity entanglement of qubits across an elementary quantum network. *Physical review letters* 124, 11 (2020), 110501.
[21] Pengfei Wang et al. 2021. Single ion qubit with estimated coherence time exceeding one hour. *Nature communications* 12, 1 (2021), 233.
[22] Kenneth Wright, Kristin M Beck, Sea Debnath, JM Amini, Y Nam, N Grzesiak, J-S Chen, NC Pisenti, M Chmielewski, C Collins, et al. 2019. Benchmarking an 11-qubit quantum computer. *Nature communications* 10, 1 (2019), 5464.
[23] Anbang Wu et al. 2022. AutoComm: A Framework for Enabling Efficient Communication in Distributed Quantum Programs. In *IEEE/ACM International Symposium on Microarchitecture*.